



Politecnico di Torino

INTEGRATED SYSTEMS ARCHITECTURE

A.Y. 2025/2026

Laboratory 3

Design and implementation of a hardware accelerator for
multichannel 1D convolution - Part 1

Delivery instructions

A template for the homework report is available on the *Portale della Didattica*. Students are required to follow the **provided structure** and ensure that the report is **self-contained, complete, clear, and accurate**. **Diagrams, graphs, and figures** that facilitate the understanding of the design must be included in the report.

Each project prepared for this course must be submitted by uploading it to the *Portale della Didattica*, section *Elaborati*. Only one member of the group must submit the deliverable.

The deliverable must consist solely of a **.pdf report**, describing both part 1 and part 2 of the laboratory. The **.pdf** must be called **isa<nn>_lab3.pdf**, where **<nn>** represents the group number (for example, group 8 should name the file **isa08_lab3.pdf**).

Delivery deadline: February 13 at 23:59

1 Introduction

The aim of this laboratory session is to design a hardware accelerator that performs a multichannel 1D convolution. This accelerator will later be implemented on *Croc*, a SoC developed by ETH Zurich and the University of Bologna¹. All the necessary files and scripts are provided in a repository via Github Classroom: <https://classroom.github.com/a/yPqdNZhI>.

2 Open-source design flow

During this lab, you will use a full open-source design flow. In particular, for the accelerator, the following tools will be used:

- **Verilator**², a SystemVerilog simulator, used for the verification of the RTL;
- **GTKWave**³, a wave viewer capable of displaying signals generated by the simulator;
- **Bender**⁴, a dependency management tool for hardware design projects, developed as a part of PULP project.

2.1 Setup Workspace

During this laboratory session, all the necessary tools will be provided within a **Docker image**.

A **Docker image** is a lightweight, standalone, and executable package that contains everything required to run a piece of software, including the code, runtime, libraries, and system tools. When a Docker image is executed, it creates a **Docker container**, which is an isolated environment where the software actually runs.

To run the Docker image, **Docker** must be installed⁵ on your PC (it is available for all major operating systems).

Depending on your operating system, one of the following scripts is available in the repository to launch a **Docker container**. (Remember: to run the script, you need to be inside the working directory.)

- `start_vnc.bat` for Windows;
- `start_vnc.sh` for macOS;
- `start_linux.sh` for Linux.

The first time you launch the script, the Docker image must be pulled rather than simply executed, so this process may take some time depending on your internet connection and system performance.

Once the container is running, it can be accessed by entering `localhost` in your browser's address bar. A password is required: `abc123`. To avoid entering the password each time, you can simply use the URL `localhost/?password=abc123`.

Thereafter, the container can be started or stopped at any time by clicking the start/stop icon in the **Containers** section of the Docker Desktop application.

2.2 Makefile

Once the Docker container is opened on your browser, the terminal can be opened by right-clicking anywhere on the desktop and selecting *Terminal Emulator*.

All the necessary tools can be executed from the terminal using the provided **Makefile**. As a first step, it is recommended to print the Makefile help message to verify that the environment is correctly set up:

```
make help
```

You should obtain an output similar to the following:

¹<https://github.com/pulp-platform/croc>
²<https://www.veripool.org/verilator>
³<https://gtkwave.sourceforge.net>
⁴<https://github.com/pulp-platform/bender>
⁵<https://docs.docker.com/desktop>

```

Conv1D Accelerator Makefile

Targets:
  all                      Default target: runs gen-data and verilator-sim

Verilator RTL Simulation:
  verilator-build          Build the simulation model without launching simulation
  verilator-sim            Build and run the simulation
  verilator-run            Run the simulation (requires prior build)
  gen-data                 Generate input data for simulation using APP_PARAMS
  waves                    Open waveform dump with GTKWave

Hardware Generation:
  gen-registers            Generate configuration registers from HJSON specification

Utilities:
  bender-update            Update vendored IPs
  bender-flist             Generate a file list of all source files for Verilator
  clean                    Remove build directory and generated data files
  clean-all               Remove all automatically generated files. Use with caution
  help                     Display this help message.

Examples:
  # Standard simulation with tracing:
  make verilator-sim LOG_LEVEL=LOG_HIGH

  # Custom test parameters:
  make gen-data APP_PARAMS="--in_len 16 --in_ch 8 --k_len 5 --k_num 4"

  # Complete workflow:
  make all

```

Before running the simulation, the CSR registers defined in *control_reg.hjson* must be generated by executing:

```
make gen-registers
```

These registers need to be regenerated only when the HJSON file is modified. If issues arise during generation, refer to Section 2.3.

Next, random input data for the simulation must be generated:

```
make gen-data
```

If no parameters are specified, data will be generated using the default `APP_PARAMS` defined in the Makefile. Since these parameters must be customized according to the design specifications, they can be set as follows:

```
make gen-data APP_PARAMS="--in_len 16 --in_ch 8 --k_len 5 --k_num 4"
```

The parameters inside the quotation marks must match the specification assigned to your design. To avoid rewriting them each time, the default values can be modified directly within the Makefile.

To build and execute the simulation using Verilator, run:

```
make verilator-sim
```

To view the waveforms generated by the simulation, execute:

```
make waves
```

2.3 Warnings



You may encounter issues when running some scripts because Windows and Unix-based systems (such as Linux and macOS) handle end-of-line characters differently. In such cases, error messages referring to problems with the carriage return character (`\r`) may appear. If a different error message is shown, it is still recommended to check the line-ending format, as this issue is often caused by incorrect carriage return handling.

This issue may affect in particular the following scripts:

- `./hw/control-reg/gen-control-reg.sh`
- `./hw/vendor/pulp\_platform\_register\_interface/vendor/lowrisc\_opentitan/util/regtool.py`

To fix this problem, an advanced text editor such as **VS Code** or **Notepad++** can be used. Ensure that the line-ending format is set to **LF** instead of **CRLF**. In most editors, this setting can be changed by clicking the line-ending indicator in the status bar, typically located at the bottom of the editor window, and selecting the appropriate format.

Another issue that may occur is that a script does not have execution permissions. When this happens, an error such as the following is typically shown:

```
<path_to_script/script>: /usr/bin/env: bad interpreter: Permission denied
```

To verify the permissions of a file, simply run:

```
ls -l <path_to_script>
```

and check whether the execution bit **x** is present.

If the execution bit is missing, permissions can be added by running:

```
chmod +x <path_to_script/script>
```

Since these errors depend on the configuration of each individual machine, they may appear on any PC working on the repository, regardless of whether they have already been fixed on another machine.

3 Understanding Multichannel 1D Convolution

A 1-dimensional convolutional layer is designed for processing sequential data such as time series or audio signals. The “1D” aspect refers to the fact that the convolution operation moves along one dimension (the sequence dimension), instead of two dimensions like in image processing. The layer becomes particularly powerful when dealing with **multichannel inputs**, which is common in tasks like audio analysis or multivariate time series.

3.1 Multichannel Convolution

In 1D convolution, **multiple channels** represent different input features. Here are some important aspects of multichannel convolution:

- **Input Channels:** This represents the number of independent feature sequences present in the input data. For instance, if you’re working with sensor data, each sensor could represent a different input channel.
- **Output Channels:** This refers to the number of filters or feature maps that the convolutional layer will learn. These output channels represent the number of new features generated after applying the convolution operation.

3.2 Example of Multichannel 1D Convolution

Imagine you have a dataset with **3 features** (e.g., temperature, humidity, and pressure) over time. These features form three independent input channels, and you want to use 1D convolution to extract useful information:

- Your input data shape would be $(in_channels, sequence_length)$. For example, $(3, 100)$:
 - $in_channels = 3$: There are 3 channels, representing different features.
 - $sequence_length = 100$: Each channel has 100 timesteps.

The convolutional layer will take **3 input channels** and apply **16 different filters**.

- The **kernel size** is 5, meaning the filter looks at a window of 5 consecutive values at each time step.
- To ensure that the **output sequence length** is the same as the input sequence length, we can use **padding** and **stride**:
 - **Padding:** Adding padding of 2 (half of the kernel size minus one) to both ends of the input sequence ensures that the filter can slide across all values without reducing the length.
 - **Stride:** Setting the **stride** to 1 ensures that the filter moves one step at a time, preserving the sequence length.

With these settings, the output shape will be $(out_channels, output_sequence_length)$, which in this example would be $(16, 100)$.

3.3 How Convolution Happens

1. **Sliding Kernel:** Each filter of size 5 slides across the input sequence, computing the dot product at each step.
2. **Multiple Filters:** Each of the output channels is produced by a different filter, and each filter looks at all input channels.
3. **Aggregation:** During convolution, the kernel applied to each input channel produces an intermediate result, and these are summed together to form the final output for that filter.
4. **Receptive Field:** Each output value is influenced by a “receptive field” consisting of multiple values from all input channels.

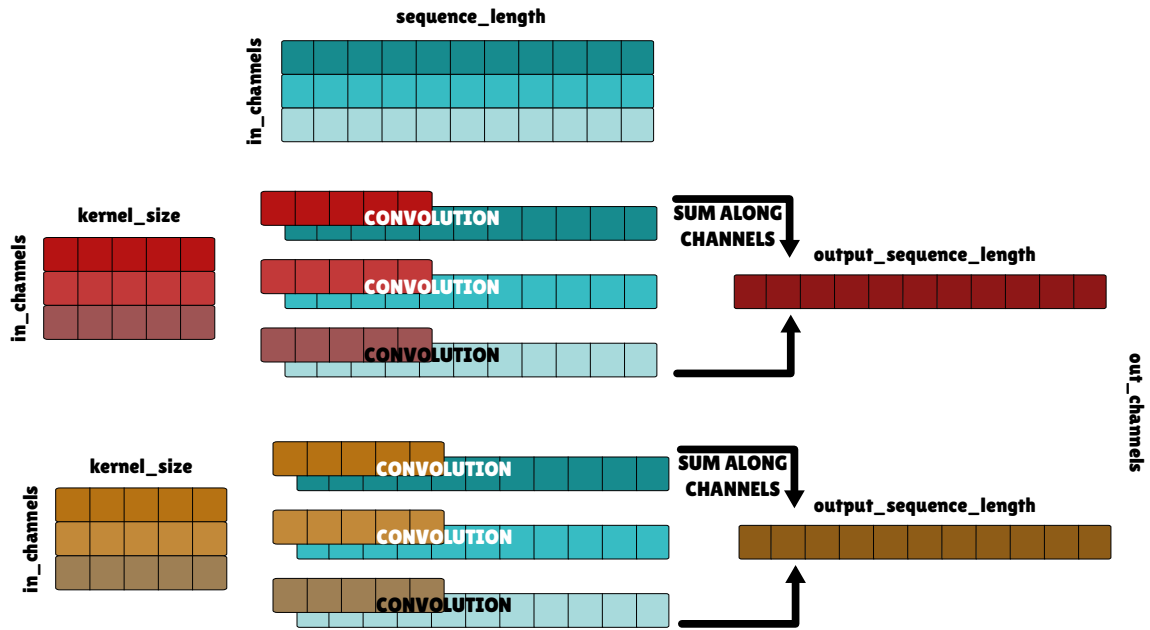


Figure 1: Example convolution with 3 input channels and 2 output channels

3.4 Why Use Multichannel 1D Convolution?

1. **Feature Extraction:** Multichannel 1D convolution is useful when extracting temporal relationships in **multivariate time series** or other sequential data.
2. **Combining Information:** It allows the model to learn relationships both within and across different input channels, making it powerful for tasks involving **multiple correlated features**.
3. **Efficient Parameter Sharing:** Using filters allows the model to efficiently share parameters across the entire sequence, reducing the number of parameters compared to fully connected layers.

3.5 Summary

- 1D convolution is designed to process sequential data, often with **multiple channels**.
- The **input channels** represent different features, and **output channels** represent the number of learned filters.
- It is widely used in applications like **audio processing**, **multivariate time series analysis**, and **natural language processing** where learning both intra- and inter-channel relationships is important.

Multichannel 1D convolution provides an efficient way to extract and learn rich temporal features from multi-feature sequential data.

4 Tiling

Tiling is a technique used to divide large computational tasks into smaller, more manageable chunks. This is done to fit data into the limited size of cache memory or specialized memory (like GPU shared memory). By **breaking the data into tiles**, the network can process parts of an image or a layer sequentially, reducing the need to continuously fetch data from slower memory locations. Tiling helps minimize latency and optimize processing by keeping data closer to where it's being used.

4.1 Input Stationary

With input stationary, the focus is on keeping the input data (like an image or an activation map) fixed in place in the processing unit's memory for as long as possible. This approach reduces the need to repeatedly load input data into memory, which saves bandwidth and energy. In convolutional neural networks (CNNs), for example, the filter slides across the stationary input to produce an output. This method is efficient when the cost of re-loading input is higher compared to storing intermediate results.

In multichannel Conv1D operations, input stationary can be particularly advantageous when dealing with sequential data that has multiple features, such as audio signals or time-series data. In a multichannel Conv1D layer, the input typically consists of several channels (features), and keeping these channels stationary helps optimize the computation by reducing redundant memory transfers while processing each feature with the filters.

4.2 Weight Stationary

Weight stationary refers to keeping the weights (or filter kernels) fixed in memory during computations. Instead of moving weight values in and out of memory frequently, the weights are loaded once and kept constant, while input data moves through them. This is particularly beneficial when the network involves small kernels that are repeatedly applied to different parts of the input. Weight stationary is useful for reducing energy consumption and the overhead associated with accessing weights multiple times.

In the context of multichannel Conv1D, weight stationary can help optimize the computation by loading the filter weights once and applying them to multiple time steps across each channel. Since the filters are often shared across different time steps, keeping them stationary ensures that the weights are reused efficiently, leading to lower memory access costs.

4.3 Output Stationary

In output stationary, the idea is to keep the output data (or partial sums) stationary in memory until all the necessary computations are completed. This means that as input data and weights are processed, the resulting partial outputs accumulate in a designated memory location, reducing the need to move data back and forth. This method is efficient in scenarios where writing output back to slower memory is a major bottleneck. Keeping the output in local memory until the computation is complete helps avoid repeated costly memory writes.

In multichannel Conv1D operations, output stationary can be beneficial when accumulating the results across multiple channels. Since the output of each convolution depends on combining data from different channels, keeping the output stationary helps in efficiently accumulating partial results without frequent memory transfers, thus improving overall computational efficiency.

5 Assignment

Design and implement an accelerator to perform multichannel 1D convolution using limited resources. The accelerator should not contain more than:

- **4 multipliers,**
- **5 adders**

The accelerator relies on a scratchpad memory of **512B** to store the inputs, weights, and outputs of the convolution.

Assume that data are stored in the scratchpad memory of the accelerator as 32 bits words. Inputs and filters are stored in 8 bit fixed-point format. The output of the convolution is stored in 32 bit fixed-point format. So at each memory location, you can store 4 inputs, 4 weights, or 1 output.

You can choose how to design the hardware to implement the algorithm. Apply the techniques you have learned in the course to optimize the design.

The accelerator should be able to run convolutions *at least* on 16 input channels, each with 128 8-bit samples, and 8 filter sets. Each filter contains 5 8-bit coefficients. The result is therefore composed of 8 output channels, each with 128 or 124 **32-bit** samples, depending on whether you choose to implement zero padding or not, respectively. Therefore, tiling may be required. You are free to choose which tiling strategy better exploits the hardware design.

Refer to the provided repositories for the complete description of the assignment and the provided files. In particular, moving in the repositories you will find **TODOs** and **README.md** files that will guide you through the assignment.

To complete the assignment, you need to:

1. **Design** the accelerator.
2. **Implement** the accelerator in **SystemVerilog**.
3. **Test** the accelerator using the **provided testbench**.